An Educational Framework for AI-Driven RF Signal Processing on FPGA

Jude Eschete[1], Bernard Yett[2] (Project Advisor)
Department of Electrical and Computer Engineering, Stevens Institute of Technology, Hoboken, NJ 07030, United States
Email: [1]JEschete@stevens.edu, [2]BYett1@stevens.edu

***Abstract*—University electrical engineering programs typically deliver field-programmable gate array (FPGA) digital design, radio-frequency (RF) communications, embedded systems, and machine learning as independent courses, leaving graduates without structured experience integrating these disciplines. This paper presents an educational framework built around a bench-top platform in which two STM32+RFM95W cooperative target stations exchange DATA and ACK traffic on a shared 906.5 MHz channel, a third STM32+RFM95W interceptor passively captures the bidirectional traffic and forwards it to a Digilent Nexys A7-100T FPGA, and collected signal data feeds a machine learning classification pipeline. A five-module progressive curriculum sequences students through hardware bring-up, FPGA RTL design, system integration and data collection, AI-driven signal analysis, and full-system validation; the first four modules were executed on hardware and the fifth is fully specified, with cohort execution left as future work. Each module makes the interfaces between disciplines explicit teaching objectives. The framework was executed end-to-end on physical hardware, producing a working ingestion pipeline, a labeled multi-campaign dataset, a four-family classifier comparison, and a live hardware inference demonstration, together with a documented set of cross-domain integration faults that constitute evidence the framework produces the multidisciplinary reasoning artifacts it is designed to teach.**

***Index Terms*—FPGA, LoRa, machine learning, RF signal processing, embedded systems, curriculum design, signal classification**

I. Introduction

Modern electrical and computer engineering practice increasingly demands fluency across multiple technical domains simultaneously. Defense, telecommunications, and aerospace systems routinely integrate real-time radio-frequency (RF) data acquisition, hardware-accelerated digital processing, embedded firmware, and intelligent signal analysis within a single pipeline. University curricula, however, frequently deliver these disciplines in isolation. A student may learn hardware description language (HDL) synthesis and timing closure in a digital design course, modulation theory and channel models in a communications course, interrupt-driven firmware in an embedded systems course, and classification on benchmark datasets in a machine learning course—without encountering a structured experience that requires synthesizing these skills into a working system.

This isolation creates a gap between academic preparation and professional practice. Engineers who understand each domain individually often struggle with the interface problems that arise when combining them: how RF modulation parameters affect field-programmable gate array (FPGA) buffer sizing, how reconfigurable logic resource constraints shape a data collection pipeline for machine learning, or how real-world RF propagation deviates from classroom channel models. These cross-disciplinary concerns are not incidental details but core engineering competencies that no single-domain course addresses.

Prior research has contributed to individual components of this problem but has not combined them into an integrated educational experience. On the pedagogical side, cloud-based FPGA toolchains with structured lab sequences have lowered the barrier to digital design education [9], and the STM32 Nucleo platform has been validated as an effective embedded systems teaching tool with significantly greater peripheral capability than Arduino-class boards [10]. Both works demonstrate that hardware-grounded, progressive lab sequences are effective within a single discipline, but neither extends that model across multiple disciplines simultaneously.

On the technical side, FPGA-based RF signal processing and inference have advanced considerably: Boolean reservoir computing achieves convolutional-neural-network (CNN)-competitive modulation classification accuracy with far fewer trainable parameters [1], streaming CNN architectures on RFSoC FPGAs classify eight modulation schemes continuously at 128 MHz [3], FPGA-accelerated ResNet reduced angle-of-arrival inference cycle count by 28.2% over a CPU baseline while improving robustness to multipath fading [2], and system-on-chip (SoC) FPGA LoRa gateways have demonstrated hardware-accelerated processing within tight resource budgets [7]. For LoRa signal classification specifically, deep learning has proven effective across multiple tasks: CNNs on spectrogram inputs achieved 99.77% spreading factor (SF) detection across a -20 dB to +30 dB signal-to-noise-ratio (SNR) range [5], CNNs on raw in-phase/quadrature (I/Q) data reached 93.3% device identification accuracy under real experimental conditions [6], neural network demodulation matched coherent matched-filter performance while tolerating carrier frequency offset [8], and comparative studies of deep neural network (DNN), CNN, and deep belief network (DBN) architectures provide empirical guidance on accuracy-complexity tradeoffs for resource-constrained deployment [4]. Each of these works advances the state of the art within its own discipline by reporting peak accuracy, throughput, or resource utilization on a curated, single-domain task. None of them is—or attempts to be—a curriculum that develops the cross-domain reasoning

skills required to build, deploy, and reason about an integrated system. What is absent is a curriculum that sequences these subsystems into a unified learning experience where the connections between them are the primary subject of instruction.

The contributions of this paper are: (1) a five-module progressive educational curriculum that integrates FPGA digital design, RF communications, embedded firmware, and machine learning into a single hardware-grounded learning sequence, with the first four modules executed end-to-end on physical hardware and the fifth fully specified for cohort execution; (2) a concrete benchtop reference platform built from commercially available components that instantiates each cross-domain interface as an experimentally observable design problem, including a button-gated PAUSE protocol that exercises closed-loop intercept-and-act behavior between the interceptor and the cooperative targets without violating Federal Communications Commission (FCC) industrial, scientific, and medical (ISM)-band rules; and (3) a documented record of cross-domain integration faults surfaced during execution—from a missing nested vectored interrupt controller (NVIC) interrupt enable that blocked end-to-end ingestion, to a synthesis-vs-simulation reset divergence that converted block RAM into discrete flip-flops, to a missing power amplifier (PA) register write that broke the link-budget monotonicity assumption, to a session-confounded frequency-error feature that perfectly predicted spreading factor for the wrong reason—each of which constitutes evidence that the framework's integration surface produces the cross-domain reasoning artifacts it is designed to teach.

The remainder of the paper is organized as follows. Section II formally defines the research problem and its context. Section III presents the framework in detail, including the hardware platform, the cross-domain frame and feature formats, the curriculum module sequence, and the pedagogical rationale. Section IV presents the measured results from the four executed modules (M1 link-budget acceptance, M2 ingestion-pipeline implementation, M3 dataset collection campaigns, and M4 classifier comparison and live inference) alongside the design-time specification for the fifth module, and closes with a direct numerical comparison against single-domain prior work. Section V concludes the paper.

II. Problem Statement

The research problem solved in this paper is the absence of a structured, progressive educational curriculum that integrates FPGA digital design, RF communications, embedded microcontroller programming, and machine learning-based signal analysis into a unified learning experience in which each cross-domain interface is a deliberate teaching objective.

The system model is a four-layer benchtop pipeline whose layers correspond one-to-one with the four target disciplines. Layer 1 is an embedded firmware layer running on STM32 microcontrollers paired with sub-GHz LoRa transceivers, which generate a bidirectional radio traffic stream on a shared ISM-band channel and host an interceptor role that passively captures that traffic. Layer 2 is an RF link layer carrying that traffic between cooperative target stations and the interceptor. Layer 3 is an FPGA ingestion layer that receives the interceptor's serialized capture, recovers framing and integrity, and emits per-emitter feature vectors. Layer 4 is a host-side machine learning layer that consumes those feature vectors to train and evaluate classifiers. The full hardware instantiation, frame and feature formats, and firmware/RTL roles are specified in Section III.

The four layers expose four cross-domain interfaces whose joint correctness defines the integrated system: a firmware-to-RF interface, an RF-to-FPGA interface, an FPGA-to-data-pipeline interface, and a data-to-model interface. The integration problem the framework targets is the structured engineering reasoning required at these four boundaries—reasoning that no single-domain course develops, because by construction a single-domain course has at most one of these interfaces as an internal concern.

The problem has a strict sequencing constraint that single-domain curricula do not impose. A student cannot meaningfully study the RF-to-FPGA interface without first having a working firmware-to-RF link, cannot study the FPGA-to-data-pipeline interface without first having a working RF-to-FPGA interface, and cannot train a useful classifier without first having a labeled dataset collected from that pipeline. Hardware bring-up must therefore come first, and every downstream learning objective inherits the correctness of what preceded it. This dependency chain is what forces a progressive module sequence rather than parallel tracks, and it is also why integration faults in any layer surface as observable failures in downstream layers—which is the property the framework exploits as its primary teaching mechanism.

The target population is senior undergraduate and master's-level electrical engineering students who have completed introductory coursework in each of the four domains individually. These students possess the foundational prerequisites—HDL syntax, basic modulation and link theory, embedded C programming, and introductory machine learning—but lack experience working with systems that require all four simultaneously, and lack the cross-domain reasoning skills required to localize an integration fault to its originating layer.

The problem being solved is therefore not a shortage of single-domain technical resources. Comprehensive textbooks, open-source FPGA IP cores, well-documented ML frameworks, and hardware reference designs exist for every component in this system. The gap is the absence of a curriculum that sequences those resources into an integrated learning experience and identifies the four cross-domain interfaces above as first-class subjects of instruction, supported by a concrete hardware platform on which integration faults can be observed, diagnosed, and traced back to their originating layer.

III. Proposed Integrated Educational Framework

The proposed solution to the integration-curriculum problem of Section II is a five-module progressive curriculum paired with a concrete benchtop hardware platform that instantiates

the four-layer system model and exposes each of its four cross-domain interfaces as a directly observable design artifact. The framework assumes foundational domain knowledge in each layer individually and targets the integration layer above them: the design decisions, interface protocols, and system-level reasoning skills that emerge only when students work across disciplinary boundaries simultaneously. The remainder of this section presents the hardware platform (§III.A), the three fixed-length data structures that make each cross-domain interface a graded engineering artifact (§III-B), the curriculum module sequence built around them (§III.C), and the pedagogical rationale that explains why this design produces multidisciplinary engineering reasoning that single-domain curricula cannot (§III.D).

A. Hardware Platform

The platform was designed around three subsystem layers, with each hardware selection driven by what it asks students to think about rather than by peak performance.

The target layer consists of two STM32 Nucleo-L476RG microcontrollers, each paired with an RFM95W LoRa transceiver operating at 906.5 MHz in the U.S. 915 MHz industrial, scientific, and medical (ISM) band. Both targets are transmit-and-receive (TX+RX) capable: Target A initiates DATA, Target B replies with ACK, and both listen continuously for PAUSE commands addressed to their respective Beacon IDs. LoRa was chosen because its chirp spread spectrum waveform exposes a small set of directly tunable parameters: spreading factor (SF), bandwidth, and coding rate. Their effects on link behavior are large enough to observe on the bench. A one-step SF increment doubles the on-air symbol time, halves the data rate, and adds approximately 2.5–3 dB of demodulator sensitivity [12]; that single radio-layer decision ripples downstream into FPGA buffer sizing, dataset collection time, and the variance of the machine-learning (ML) training set, a chain of consequences spanning all four disciplines and recorded empirically by the C2 sweep at §IV-C. The RFM95W was selected over alternative sub-GHz transceivers for its 3.3 V serial peripheral interface (SPI), 915 MHz ISM operation, and support for both LoRa and frequency-shift keying / on-off keying (FSK/OOK) modulations. Prior work confirms that LoRa hardware produces RF signatures rich enough for device identification [6] and neural network demodulation [8], validating LoRa as a suitable signal source for the full pipeline [10], [13].

The interceptor (Threat) layer uses a third STM32+RFM95W node, hardware-symmetric with the two targets but operated under a different firmware role. The Threat passively captures every demodulable frame on the shared channel, maintains a per-Beacon-ID received signal strength indication (RSSI) observation table, and, while its onboard user button is held, transmits a PAUSE packet at a 100 ms cadence addressed to the highest-RSSI station, with each PAUSE carrying a 500 ms hold-off in its payload. The PAUSE packet uses the same 32-byte radio frame format as DATA and ACK (§III-B), so its existence does not change the FPGA ingestion pipeline or the dataset schema. The hardware symmetry between the Threat and the targets is itself a teaching point: a single-domain RF receiver design would have no reason to be symmetric with the transmitters, but the framework's intercept-and-act loop requires the third radio to be capable of both reception and well-formed transmission on the same channel.

At the processing layer, the Digilent Nexys A7-100T serves as the FPGA development platform. Its Artix-7 fabric, with 240 digital signal processing (DSP) slices, approximately 4,860 Kb of block RAM (BRAM), and 63,400 lookup tables (LUTs) [11], is large enough to implement a complete signal-processing datapath but constrained enough that students must make deliberate resource allocation decisions; the measured Module 2 implementation in §IV-B exercises that constraint directly. The board connects to the STM32 interceptor through its peripheral module (PMOD) headers, giving students a concrete physical interface to characterize and debug. Prior work has shown that comparable inference pipelines fit within the resource envelope of similar devices [1], [3], confirming the Artix-7 is sized to make tradeoffs visible without precluding the full pipeline.

At the analysis layer, a host machine accumulates the labeled feature vectors exported by the FPGA and serves as the ML development environment, with prior comparative results on RF modulation classification [4] and LoRa-specific deep learning benchmarks [5] as reference points.

B. Cross-Domain Frame and Feature Formats

The four cross-domain interfaces identified in Section II are made concrete by three fixed-length data structures that every student implements: a 32-byte radio packet (DATA, ACK, or PAUSE) emitted by the STM32 firmware, a 43-byte forwarding frame wrapped around it by the STM32 interceptor before handoff to the FPGA, and a 32-byte feature vector produced by the FPGA and consumed by the downstream machine learning classifier. These artifacts are the primary instructional objects of the framework: every field in each one corresponds to a design decision made in one disciplinary domain and observed in another.

Table I shows the radio packet. Its on-air metadata fields (Beacon ID, sequence number, packed modulation code, spreading factor, transmit timestamp) serve a dual role: they give the downstream ML pipeline supervised ground-truth labels for emitter ID, packet type, modulation kind, and spreading factor, and they let the FPGA cross-check its own detection logic against values the transmitting station claims it sent. Transmit power and bandwidth are configured at the SX1276 register level by `lora_init()` and recovered downstream from the campaign configuration rather than from any packet field. The trailing CRC-16/CCITT-FALSE distinguishes link errors from downstream corruption.

The interceptor wraps each captured radio packet in a 43-byte forwarding frame that prepends a start delimiter, length, RX timestamp, and three SX1276-derived measurement fields

TABLE I
RADIO PACKET FORMAT (32 BYTES; SAME LAYOUT FOR DATA, ACK, AND PAUSE), AS EMITTED BY `BUILD_PACKET()` IN THE REFERENCE FIRMWARE.

Field	Offset	Size
Preamble sync (`0xAE`, `0x5A`)	0	2 B
Beacon ID (sender)	2	1 B
Sequence number (LE)	3	2 B
Payload length (fixed `0x0E`)	5	1 B
Modulation code (kind$\ll$2 | type)	6	1 B
Spreading factor (LoRa only)	7	1 B
Reserved (`0x00`)	8	1 B
TX timestamp, ms (LE)	9	4 B
Payload (DATA/ACK) or PAUSE args	13	17 B
CRC-16/CCITT-FALSE (BE)	30	2 B

TABLE II
FEATURE VECTOR FORMAT (32 BYTES, ONE BRAM ROW).

Field	Offset	Size
Beacon ID	0	1 B
Modulation code (kind only)	1	1 B
Spreading factor	2	1 B
`pkt_type` (DATA/ACK/PAUSE)	3	1 B
RSSI (latest)	4	2 B
SNR (latest)	6	2 B
Frame count	8	4 B
Last arrival timestamp	12	4 B
Inter-arrival time	16	4 B
RSSI delta	20	4 B
Priority score	24	4 B
`target_id_if_pause`	28	1 B
`pause_duration_units`	29	1 B
Reserved / padding	30	2 B

(RSSI, SNR, frequency error) that are not available to the originating station. The radio packet rides verbatim as the payload, and a frame-level CRC-8 protects the transport between the STM32 and the FPGA independently of the inner LoRa CRC.

Table II shows the FPGA feature vector, a 32-byte row indexed by Beacon ID in a 256-row dual-port BRAM. Three of its fields (inter-arrival time, RSSI delta, priority score) are computed on the FPGA from per-station state registers rather than copied from the forwarding frame, making the feature vector the first point at which the pipeline begins to abstract raw packet contents into classifier-ready features. The format is fixed at module-boundary specification time so that Module 2 (FPGA feature extraction) and Module 4 (host classifier training) can be developed against the same contract without either blocking on the other.

The separation of concerns between these formats is itself instructive: the radio packet is the object the originating firmware owns and the ML pipeline labels against, while the forwarding frame is the object the interceptor and FPGA ingestion datapath own. A single dual-flip-flop synchronizer on `uart_rxd` handles the only intentional clock-domain crossing in the design, and Vivado's CDC analyzer correspondingly returns an empty report on the routed implementation (§IV-B). A student who changes the radio packet layout without coordinating the interceptor's payload offset, or who introduces a second clock domain without an explicit synchronizer, produces exactly the kind of silent cross-domain failure the framework is designed to surface.

C. Curriculum Module Sequence

Module 1: RF Hardware and Subsystem Design. Module 1 spans eight lab handouts (LH1-A through LH1-H) and walks students from loose components to a working end-to-end RF→firmware→FPGA chain. Students assemble the RFM95W breakout and verify its idle current draw (LH1-A), set up the STM32 development toolchain and validate it with a debugger-driven blinky (LH1-B), bring up SPI between the STM32 and the SX1276 by reading `RegVersion` = `0x12` and exercising radio mode transitions (LH1-C), and demonstrate a bidirectional DATA/ACK ping-pong baseline between two cooperative targets with measured packet error rate (PER), RSSI, and SNR (LH1-D). They then validate the FPGA toolchain on the Nexys A7 with switch-to-LED and UART-echo loopback designs (LH1-E), promote the LH1-D skeleton into production cooperative-target firmware structured around an explicit finite state machine (FSM)—`INIT`→`CONFIG`→`IDLE`→`BUILD`→`TX`→`WAIT_ACK`→`LOG`—that includes a shared `PAUSED` branch entered on receipt of an addressed PAUSE packet (LH1-F), implement the interceptor (Threat) firmware with `RxDone` interrupt-service-routine (ISR)-driven capture into a ring buffer, the per-Beacon-ID RSSI observation table, the button-gated PAUSE TX scheduler, and a 43-byte CRC-protected forwarding frame on USART3 (LH1-G), and finally wire the interceptor's USART3 TX into the Nexys A7 PMOD and write a five-source-file VHDL ingestion skeleton whose seven-segment display acts as a live frame counter validating the complete RF→interceptor→FPGA path (LH1-H). The primary learning objective is understanding how firmware design choices—SPI clock rate, DMA configuration, interrupt latency, half-duplex TX/RX role switching, and ISR-vs-main-loop work partitioning—propagate through the firmware-to-RF and RF-to-FPGA interfaces of §II and determine the timing and integrity of the data stream the FPGA ultimately consumes. The Nucleo-L476RG's multiple independent serial interfaces and hardware-accelerated peripherals [10] provide enough configuration space to make these tradeoffs concrete and experimentally observable.

Module 2: FPGA Register-Transfer-Level (RTL) Design and Simulation. Module 2 spans eight lab handouts (LH2-A through LH2-H) that incrementally replace each block of the LH1-H bring-up skeleton with a verified production version, then close the design through implementation, timing analysis, and a signed-off bitstream. Students open the LH1-H `ingest_top` project and add a simulation fileset, two recorded interceptor captures (`capture_clean.bin`, `capture_mixed.bin`) staged into a `data/` folder, and a one-line passthrough that routes the FPGA's PMOD UART input directly to the Nexys A7's onboard FT2232 USB-UART so the interceptor's byte stream can be captured on the same cable that programs the board (LH2-A). They

then write a generic-parameterized 115,200-baud `uart_rx` with a one-cycle `rx_strobe` per received byte plus a unit testbench and a regression that replays every byte of `capture_clean.bin` (LH2-B), widen the `byte_fifo` (a first-in-first-out, or FIFO, buffer) from 16 to 64 entries with overflow stress coverage (LH2-C), and replace the LH1-H `frame_detector` with a packet-parser FSM that locks onto the 0x7E start delimiter, walks the 1-byte length, the 8-byte interceptor metadata, the 32-byte embedded radio packet, and the trailing CRC-8, and emits registered field outputs on `frame_valid` (LH2-D). Two single-cycle-per-byte CRC engines (CRC-8 over forwarding-frame bytes 1–41, CRC-16/CCITT-FALSE over embedded-radio-packet bytes 0–29) are then wired into the parser to gate `frame_valid` on integrity (LH2-E), each validated frame is transformed into a fixed 32-byte feature vector indexed by Beacon ID and stored in a dual-port BRAM with a host read port (LH2-F), an end-to-end testbench drives the full pipeline (`uart_rx` → `byte_fifo` → `pkt_parser` → `feature_extract` → `feature_bram`) against three recorded captures and compares each exported feature vector byte-for-byte against an independent Python reference (LH2-G), and the module closes by running implementation, analyzing the post-route resource utilization, timing slack, and clock-domain-crossing (CDC) reports, and producing the signed-off bitstream that Module 3 deploys (LH2-H). Students encounter FPGA-specific design constraints—timing closure, clock-domain crossing, memory resource allocation, BRAM-vs-distributed-RAM inference, and synchronous-reset side effects on storage primitives—in the context of a real signal protocol rather than a synthetic exercise. Cloud-based FPGA toolchains have demonstrated that structured, hardware-grounded lab sequences can make this learning accessible without requiring specialized hardware beyond a web browser [9]; this module follows that model while adding the RF-protocol context and the upstream-firmware coupling that a standalone digital-design course omits.

Module 3: System Integration and Data Collection. Module 3 spans eight lab handouts (LH3-A through LH3-H) that take the verified subsystems of Modules 1 and 2, integrate them into a single live pipeline, characterize its reliability and timing, and execute a structured multi-campaign data-collection program whose output is the data contract for Module 4. Students first verify every Module 1 and Module 2 subsystem in isolation against a signed-off pre-integration checklist and produce the wiring plan that will serve as ground truth for downstream debugging—deliberately deferring all cross-board connections until single-subsystem correctness is established (LH3-A). They then instrument the `ingest_top` design with Vivado integrated logic analyzer (ILA) probes and generate a separate debug bitstream that becomes the primary diagnostic tool for the rest of the module (LH3-B). With the ILA armed, they connect the STM32 interceptor's USART3 TX to the Nexys A7 PMOD and work through a four-layer debugging sequence (physical interface, baud rate and framing, protocol synchronization, data integrity) until the ILA captures the first `frame_valid` pulse on live hardware—the central Module 3 milestone, validating an end-to-end path that spans all four hardware components and all four cross-domain interfaces of §II simultaneously (LH3-C). They then quantify packet error rate decomposed by pipeline stage and verify a closed-form end-to-end latency budget on hardware at two spreading-factor settings (LH3-D), characterize sustained-throughput behavior under cooperative-target ping-pong with optional PAUSE-burst load and produce a separate production bitstream (without the ILA, so its block-RAM cost is reclaimed for downstream collection runs) used for all subsequent data collection (LH3-E). The collection program is a host-side Python script that receives the interceptor's 43-byte forwarding frames through the FPGA's onboard USB-UART bridge, parses the embedded radio packet fields, attaches ground-truth labels from a per-run JSON campaign configuration, and writes labeled records to a CSV file; students execute the TX-power sweep (C1) and spreading-factor sweep (C2) under this script (LH3-F), then complete the controlled experimental dataset with the distance sweep (C4) and the active-station-count scaling sweep (C5) including a two-station ping-pong configuration (LH3-G). The module closes with the merge step: all per-campaign CSVs are combined into a single labeled dataset against the data dictionary that becomes the Module 4 data contract, five required exploratory data analysis visualizations are produced, and the operating-condition shift between the clean single-station regime (C1, C2, C4) and the contended multi-station regime (C5) is documented entirely on hardware data, generalizing the well-known sim-to-real distribution-shift result [6] to a fully on-hardware setting (LH3-H). Synchronization mismatches and protocol errors that were invisible in per-subsystem development surface during integration as cross-domain failures that must be diagnosed and resolved against the ILA waveforms and the per-stage PER decomposition—which is the integration-debugging skill the module exists to develop.

Module 4: AI-Driven Classification Pipeline. Module 4 spans eight lab handouts (LH4-A through LH4-H) that take the merged dataset produced by LH3-H, train and compare four classifier families (random forest, support vector machine (SVM), multi-layer perceptron (MLP), and one-dimensional (1-D) convolutional neural network (CNN)) per task, characterize their robustness across operating conditions, and select a deployable model per task that runs against the live FPGA stream. The module opens with a nine-check data-integrity audit run before any preprocessing or training is attempted—schema, row count, nulls, duplicates, valid ranges, RSSI sign, label consistency, class balance, and split integrity—which establishes the data contract the rest of Module 4 depends on (LH4-A). Students then preprocess into model-ready matrices: drop join-mid-stream artifact rows, encode labels, select task-specific feature sets that explicitly avoid label leakage (with a logistic-regression sanity check that trains on *forbidden* columns alone and must hit $\geq$0.95 to confirm the forbidden list correctly identified the leak vectors), and fit a `StandardScaler` that downstream

classifiers reuse, producing reusable `Preprocessor` artifacts and `X_train.npy`/`y_train.npy` bundles for the four classification tasks (SF, modulation, emitter ID, packet type) (LH4-B). A three-rung classical baseline (majority-class `DummyClassifier`, fixed-depth decision tree, grid-searched random forest) is established per task, with the random-forest accuracy serving as the bar every later model must beat (LH4-C). Students then train an SVM classifier with linear and radial-basis-function (RBF) kernels and run a permutation-importance analysis that gives a model-agnostic feature ranking trustworthy for correlated features where Gini importance is misleading (LH4-D), implement a small fully-connected MLP ($F \to 64 \to 32 \to C$) in PyTorch with early stopping and dropout to test whether added representational capacity helps on the tabular feature set (LH4-E), and build a 1-D convolutional network that consumes short *sequences* of feature vectors and run the framework's defining operating-condition shift experiment: train on the clean regime (single-station C1/C2/C4) and evaluate on the contended regime (multi-station C5), generating a within-hardware distribution-shift gap that is the framework's capstone machine-learning result [6] (LH4-F). They then produce the three stratified-accuracy curves (SNR, distance, active-station count) that describe how each classifier degrades as operating conditions worsen and that supply the robustness evidence consumed by the final model-selection rubric (LH4-G), and the module closes by assembling a reusable `InferenceEngine`, running a 1,000-call latency microbenchmark per candidate, mechanically scoring every candidate against the deliverable rubric using only values read from the LH4-A through LH4-G manifest files (no hand-tuning), picking the deployed model per task, and streaming live predictions from the FPGA's UART export to demonstrate end-to-end inference on hardware (LH4-H). Streaming inference architectures [3] and FPGA-accelerated ML pipelines [2] serve as reference designs for students considering on-device deployment, and comparative architecture studies [4] together with LoRa-specific classification results [5], [6], [8] provide empirical benchmarks against which students evaluate their own model performance.

Module 5: System Validation and Benchmarking. Module 5 spans eight lab handouts (LH5-A through LH5-H) that derive the system specification empirically rather than asserting it a priori, document the framework's cross-domain integration learning as a structured artifact, and package the curriculum for delivery by another instructor. Students write the formal stress-test plan and execute its first dimension—S1, active-station density—to determine the empirical maximum number of concurrent stations the integrated system supports without unacceptable PER degradation or FIFO overflow, producing the first row of the Module 5 specification table (LH5-A). They then execute three of the remaining stress dimensions (S2: dynamic SF tracking by the deployed three-task classifier suite while a second target remains fixed; S3: low-SNR sweep extending the LH4-G accuracy curve plus an out-of-distribution obstruction test; S4: 30-minute endurance run that surfaces time-dependent failure modes), with the dropped `mod` task explicitly excluded since it is single-class in the dataset (LH5-B). They instrument the full pipeline—Target A TX start through Threat interceptor classification output—with a six-stage general-purpose-input/output (GPIO)-toggle latency decomposition that extends the LH3-D single-stage GPIO method to every pipeline stage, and run the S5 PAUSE behavioral-response stress test that measures the closed-loop latency from a held-button assertion at the Threat to the moment a Target prints `I'm Jammed` on USART2, plus the recovery latency once the button is released (LH5-C). They then run the six benchmarking operating points, compare the stressed-system performance against the Module 3 and Module 4 baselines, and fill in the single-page system specification sheet that any future user can refer to for operating envelope, three-task accuracy, expected latency, and PAUSE behavioral-response latency (LH5-D). The module's most explicit pedagogical statement comes next: students assemble the structured catalog of every failure or degradation mode observed across Modules 1–5, trace each entry back to its originating design decision, and produce the cross-domain dependency diagram visualizing how a decision in one module propagated into a failure in another (LH5-E). The remaining handouts are deliverable-focused: LH5-F packages the complete self-contained curriculum and instructor's guide that allows a future instructor to deliver the framework without contacting the original developer, LH5-G structures the writing of the final report and enforces the IEEE conventions and Discussion-section bar, and LH5-H prepares the 20-minute capstone presentation with a live three-task classification demonstration and the button-gated PAUSE behavioral response, plus submission of the complete curriculum package. The benchmarking and stress-test work requires active engagement across all four cross-domain interfaces of §II simultaneously: RF link budget at S3, FPGA timing and resource margin at S1 and S4, serial interface throughput at S1, ML classification accuracy under shift at S2 and S3, and the closed-loop PAUSE response latency at S5—which together constitute the most concentrated cross-domain integration exercise in the curriculum.

D. Pedagogical Rationale

The framework's central design principle is that the interfaces between disciplinary domains are the primary teaching objects, and this principle gives it concrete advantages over existing single-domain curricula. Prior work has demonstrated that hardware-grounded, progressive lab sequences are effective within a single discipline [9], [10]; this framework extends that validated model to four disciplines simultaneously. The key difference is that integration failures—a missing firmware NVIC interrupt enable that prevents the interceptor's DMA-complete callback from ever firing and silently throttles end-to-end ingestion to one frame per board reset, an FPGA synchronous-reset clause on a 256×256-bit storage array that converts what should infer as block RAM into 65,536 discrete flip-flops and shifts the implementation's resource profile by an order of magnitude, a single missing PA register write that breaks the link-budget monotonicity assumption baked

TABLE III
MODULE 1 ASSESSMENT RUBRIC (REPRESENTATIVE).

Deliverable	Weight	Due
RF module trade study report	20%	Session 1
Hardware requirements document	20%	Session 2
Cooperative target subsystem design	30%	Session 3
Interceptor subsystem and end-to-end	30%	Session 4

into a campaign's validation script, or a session-confounded transceiver frequency-error feature whose perfect correlation with spreading factor lets a classifier reach 100% test accuracy for entirely the wrong reason—are not obstacles to be avoided but structured learning events. Each one forces the student to reason across domain boundaries in a way that no isolated course can replicate, and each one is a real fault observed during execution of this framework on the platform of §III.A.

The framework is also designed to produce a category of evidence that purely accuracy-driven approaches do not. Prior work on LoRa signal classification consistently reports peak accuracy on curated datasets [5], [6], [8] but does not report root-cause attribution across module boundaries, because those traces presuppose an integrated system that the cited works are not designed to produce. By instrumenting every module with measurement scaffolds whose outputs are consumed by the next module, by requiring per-disciplinary-interface acceptance gating at every module boundary, and by treating each integration fault as a graded teaching artifact with its originating layer explicitly identified, the framework makes integration the graded outcome rather than a side effect.

Because the framework is intended to be adopted by instructors who may not share the author's depth in all four domains, the assessment structure is specified as concretely as the hardware. Table III shows the Module 1 assessment breakdown as a representative example: each per-session deliverable exercises one or more of the cross-domain interfaces of Section II, and the weights reflect the difficulty of the integration work rather than the volume of code produced. The trade study and hardware requirements document together account for 40% of the module grade even though they contain no executable code, because the framework treats the *justification* of a cross-domain design decision as a first-class graded artifact. This rubric structure is reproduced with analogous weights in each of the remaining four modules.

Finally, the framework is designed for reproducibility and transferability. All components—the Nexys A7-100T, the STM32 Nucleo-L476RG, and the RFM95W—are commercially available. Prior work has validated that the core technical pipelines are feasible within these hardware resource budgets [1], [3], [7], meaning an instructor at another institution can deploy the same curriculum without custom hardware or proprietary infrastructure. The framework's contribution is not a novel algorithm or hardware design but a structured, teachable model of how these four engineering disciplines connect at the system level—with each connection made concrete through implementation on real hardware.

IV. NUMERICAL RESULTS AND ANALYSIS

This section presents the measured quantitative results from executing the first four modules of the framework on physical hardware, paired where applicable with the corresponding closed-form prediction. Module 5 was specified during the project but not run on hardware due to schedule constraints; its design content is summarized in §III.C and its envelope claims are deferred to future cohort execution. Section IV.A presents the closed-form RF link budget that frames the Module 1 physical-layer expectations. Section IV.B reports the measured Module 2 ingestion-pipeline implementation on the Nexys A7-100T (utilization, timing, CDC, power, and the eight-testbench verification chain). Section IV.C reports the Module 3 end-to-end measured campaign yields and dataset integrity. Section IV.D reports the Module 4 four-family classifier comparison with Wilson-interval reporting, the operating-condition shift result, and the live hardware inference demonstration. Section IV.E briefly notes the design-time Module 5 specification scope. The section closes with a direct numerical comparison against published single-domain LoRa spreading-factor classification work [5].

A. RF Link Budget and Expected RSSI

The Module 1 baseline configuration (Lab Handout 1-D) operates the SX1276-based RFM95W at $f_c = 906.5$ MHz, spreading factor SF7, bandwidth $B = 125$ kHz, coding rate 4/5, and transmit power $P_{\mathrm{tx}} = +14$ dBm into a 915 MHz helical spring antenna. The free-space path loss for isotropic radiators is

$$L_{\mathrm{FSPL}}(d) = 20\log_{10}\left(\frac{4\pi d f_c}{c}\right), \tag{1}$$

which at $d = 1$ m evaluates to 31.59 dB at 906.5 MHz. Assuming realistic on-breadboard spring-antenna efficiency losses of $G_{\mathrm{tx}} = G_{\mathrm{rx}} \approx -2$ dB and no additional obstruction, the expected received power is

$$P_{\mathrm{rx}} = P_{\mathrm{tx}} + G_{\mathrm{tx}} + G_{\mathrm{rx}} - L_{\mathrm{FSPL}}(1\,\mathrm{m}) \approx -21.6\ \mathrm{dBm}. \tag{2}$$

Against the SX1276 LoRa sensitivity floor of -123 dBm at SF7 and $B = 125$ kHz [12], the SF7 operating point is expected to leave $\sim$101 dB of link margin at 1 m bench separation, and the longest configured spreading factor (SF12) is expected to retain the same margin plus an additional $\sim$13 dB of processing gain [5], [12]. This wide expected margin is what makes SF7 the correct pedagogical default: students should observe clean packet reception and valid CRCs without a range environment, and the operational geometry can later be moved away from the 1 m bench reference (the C1 sweep of §IV-C was conducted at 3 m, with measured RSSIs at +14 dBm in the -71 dBm range, consistent with the additional path loss expected at the larger separation) to recreate marginal-link behavior without changing the protocol stack.

TABLE IV
MODULE 2 INGESTION PIPELINE POST-SYNTHESIS UTILIZATION, TARGET `xc7a100tcsg324-1`, VIVADO 2025.2. THE "PROJECTED" COLUMN IS THE PER-BLOCK SUM ASSUMING CLEAN BRAM INFERENCE; SEE PROSE FOR THE SYNTHESIS-FAULT EXPLANATION OF THE GAP.

Resource	Projected	Measured	Avail.
Slice LUTs	∼600	19,161	63,400
Slice Registers	∼640	53,032	126,800
Block RAM (36-Kb tile)	5	0	135
DSP48 slices	0	0	240
Bonded I/O blocks	49	49	210
BUFGCTRL	1	1	32
Util. (LUT/FF/BRAM)	—	30/42/0%	—

TABLE V
MODULE 2 POST-ROUTE TIMING, CDC, AND POWER SUMMARY AT 100 MHZ.

Metric	Value
Worst negative slack (setup)	+1.191 ns
Worst hold slack	+0.030 ns
Failing setup endpoints	0 / 105,975
Failing hold endpoints	0 / 105,975
CDC report entries	0 (single clock domain)
Total on-chip power	236 mW
Dynamic	139 mW
Static	98 mW
Junction temperature (still air)	26.1 °C

B. Module 2 Measured Implementation: Resource, Timing, Power, and Verification

Module 2 produces the complete FPGA ingestion pipeline whose specification was given in §III-B: dual-flip-flop synchronizer on `uart_rxd`, 115,200 8N1 `uart_rx` with one-cycle `rx_strobe`, 64-deep `byte_fifo`, packet-parser FSM with single-cycle-per-byte CRC-8 and CRC-16/CCITT-FALSE engines, `feature_extract` datapath, 256-row × 256-bit `feature_bram` indexed by Beacon ID, and host UART export. The full pipeline was synthesized, implemented, and signed off in Vivado 2025.2 against the `xc7a100tcsg324-1` target on the Nexys A7-100T board, and was independently verified against a Python reference through ten testbenches.

Table IV reports the post-synthesis utilization. The headline measurement is the substantial gap between the closed-form expectation (∼600 LUTs, ∼640 FFs, 5 BRAM tiles) and the actual implementation: 19,161 LUTs, 53,032 FFs, and 0 BRAM tiles. The cause is direct and instructive: the synchronous-reset clause added to `feature_bram` during simulation bring-up to deterministically clear stale read values asks Vivado to clear 256 rows × 256 bits in a single cycle, which the BRAM hardware primitive does not support, so synthesis falls back to flip-flop inference for the entire 65,536-bit storage. The simulation worked because xsim treats the array as a generic signal; synthesis is forced to take the structural meaning literally. This is the third of the four documented integration faults of §III.D and is exactly the kind of cross-tool friction the framework is designed to surface. The standard remediation is to drop `mem` from the reset clause and rely on Vivado's BRAM init-to-zero behavior, which restores the expected 2 BRAM tiles and a few-thousand-flop count; this is logged as a Module 3 polish task rather than a Module 2 blocker because the design closes timing as-implemented.

Table V reports the post-route timing, CDC, and power summary at the target 100 MHz system clock. All 105,975 timing endpoints meet both setup and hold; the worst negative slack is +1.191 ns and the worst hold slack is +0.030 ns. The critical path runs from `u_feat/bram_wr_data_reg[3]` through 8.428 ns of routing (89.7% of which is wire delay) into a clock-enable on `u_bram/mem_reg[155][102]`—a fanout pattern that is a direct consequence of the BRAM-as-flip-flops result above, since the 65,536-flop storage is spread across hundreds of slices instead of being compactly inside two BRAM tiles. Vivado's CDC analyzer reports zero clock-domain crossings: the only intentional CDC is the dual-flip-flop synchronizer on `uart_rxd`, and Vivado classifies that input as a static asynchronous input rather than a second clock domain because it has no input-delay constraint. Total on-chip power is 236 mW (139 mW dynamic + 98 mW static), within the original <200 mW dynamic envelope despite the resource bloat, because the bulk of the inferred storage flops are quiescent between frame events and contribute under 1 mW of register power.

TABLE VI
MODULE 2 VERIFICATION CHAIN. THE `DIFF` COLUMN IS THE BYTE-DIFFERENCE COUNT BETWEEN HARDWARE DUMP AND PYTHON REFERENCE FOR LH2-G OR PASS FOR UNIT TESTBENCHES.

#	Testbench	Result
1	`tb_uart_rx` (single byte)	PASS
2	`tb_uart_rx_regression` (62,952 B)	PASS, 0 mismatches
3	`tb_byte_fifo`	PASS
4	`tb_byte_fifo_overflow`	PASS
5	`tb_pkt_parser`	PASS
6	`tb_crc` (golden vector)	PASS
7	`tb_feature_extract`	PASS
8	`tb_ingest_top` (clean capture)	0-byte `diff`
9	`tb_ingest_top` (mixed capture)	0-byte `diff`
10	`tb_ingest_top` (corrupt capture)	0-byte `diff`

The verification deliverable is the ten-testbench regression chain of Table VI, run on the framework's own implementation. All ten passed; the three full-pipeline scenarios of LH2-G (`tb_ingest_top` clean, mixed, and corrupt) produced byte-for-byte agreement with an independent Python reference across all 256 BRAM rows for every capture, including a synthesized variant of `capture_mixed.bin` with ∼5% of frames corrupted by single-bit flips. The corrupt-stream agreement is the strongest single check: independent VHDL and Python implementations of the CRC-8 and CRC-16/CCITT-FALSE polynomials, run against the same bit-flipped frames, agree on every accepted-vs-rejected decision and on every accumulated feature vector with zero discrepancies.

TABLE VII
MODULE 3 MEASURED CAMPAIGN YIELDS. PER-RUN PACKET COUNTS ARE 200 (C1, C2) AND 300 (C5). C1 STEPPED TX POWER ACROSS +2, +8, +14, AND +20 DBM; C2 STEPPED SPREADING FACTOR ACROSS SF7–SF10.

Camp.	Sweep	Runs	Rows	Status
C1	TX power	4	800	Complete
C2	Spreading factor	4	800	SF11/12 abandoned
C3	Modulation	0	0	Removed
C4	Distance	0	0	Deferred
C5	Station count	2	600	C5a/C5b only
Total	—	10	**2,200**	—

C. Module 3 Measured Dataset Yields and Integrity

Module 3 executed the data-collection campaigns of §III.C and produced the merged labeled dataset that becomes the Module 4 input contract. Table VII reports the as-collected campaign yields. Three deviations from the original five-campaign plan are documented honestly: the modulation-type sweep (C3) was removed during curriculum revision because the platform was operated LoRa-only for this iteration, the distance sweep (C4) was deferred under schedule constraints, and the spreading-factor sweep (C2) was scoped to SF7–SF10 because supporting SF11/SF12 required three additional SX1276 register writes (`RegDetectOptimize`, `RegDetectionThreshold`, `LowDataRateOptimize`) and a longer TX deadline that there was insufficient time to validate. The active-station-count sweep (C5) was executed at sub-configurations C5a (1 station, single-stream baseline) and C5b (2 stations, ping-pong); C5c–e were not run.

C1 produced a textbook monotonic mean-RSSI trend across +2/+8/+14 dBm at -82.25 / -75.92 / -71.23 dBm respectively (200 packets per level, $\sigma \leq 0.86$ dB at all three points), consistent with link-budget expectations within ± 1 dB step linearity. The +20 dBm point inverted the trend at -77.42 dBm, $\sim$6 dB below the +14 dBm anchor with otherwise normal SNR, sequence integrity, and frequency error. Root cause was identified as the missing `RegPaDac = 0x87` register write that the SX1276 datasheet requires for outputs above +17 dBm: with PA_BOOST selected via `RegPaConfig` but `RegPaDac` left at the default `0x84`, the PA enters compression at a level below +14 dBm. The +20 dBm CSV is retained in the dataset as documented evidence of this single-register-write firmware bug rather than rerun after a silent fix—the cross-layer issue is itself the second of the four documented integration faults of §III.D and is one of the framework's intended teaching moments. C2 produced clean SF7–SF10 sweeps with the expected per-step processing-gain progression in mean SNR (9.59 / 11.84 / 12.79 / 13.14 dB across SF7→SF8→SF9→SF10), confirming the demodulator behavior closed-form theory predicts. C5a and C5b produced clean baselines with zero FIFO overflow on the Nexys A7 over a 5-minute timed run.

The merge step combined the ten per-campaign CSVs into a single dataset of 2,197 rows after duplicate removal (3 rows dropped). The nine-check LH4-A data-integrity audit passed all checks (schema, row count, nulls, duplicates, valid ranges, RSSI sign, label consistency, class balance, split integrity), with zero DATA-row label-vs-self-report mismatches confirming the firmware fix from the C2 SF8 onward held across the entire corpus. One pedagogically important EDA finding emerged: `freq_error_hz` correlates with `spread_factor` at $r = 1.000$ in this dataset, because each SF was collected in a separate session and the SX1276 crystal frequency drifts $\sim$131 Hz/session as die temperature stabilizes, so the C2 sweep runs in SF order produced a frequency-error trend perfectly aligned with the SF axis. Any classifier using `freq_error_hz` for the SF task would thus succeed for the wrong reason—the fourth of the four documented integration faults of §III.D. The column is added to the SF task's forbidden list at LH4-B, and a leak-only logistic-regression sanity check confirms 1.000 forbidden-only test accuracy on the SF, beacon, and packet-type tasks (validating the forbidden-list construction).

The run-level stratified split was degenerate at this campaign volume: each minority SF class (SF8, SF9, SF10) had only one collection run, so the run-level splitter assigned all of them to the training partition and produced single-class validation and test sets. The merge script detects this condition and falls back to a row-level stratified split keyed on `label_sf`, producing 1,538 / 330 / 329 train/val/test rows and ensuring every label class is present in every split. The fallback is logged with a warning describing the leakage trade-off (within-run channel state can leak across splits), and the framework documents the long-term fix as collecting additional minority-class runs in fresh sessions to restore run-level safety.

D. Module 4 Measured Classifier Results, Operating-Condition Shift, and Live Inference

Module 4 trained four classifier families on each of the three deployable tasks (`sf`, `beacon`, `pkt`; the `mod` task is single-class in the LoRa-only dataset and was skipped). Table VIII reports the headline test accuracy and 95% Wilson confidence interval per (task, model). The `sf` task is the only one with non-trivial discrimination: the 1-D CNN over 8-frame sequences reached 0.985, a $\sim$2-percentage-point improvement over the random forest, SVM, and MLP that all tied at 0.963–0.966 within Wilson-CI overlap. The MLP seed-robustness sweep (seeds 0–3) produced $\sigma = 0.002$ on `sf`, well below the 0.02 stability bar. The `beacon` and `pkt` tasks score 1.000 on every model because of a structural alias in the C5b ping-pong design where every Target B row is an ACK and every Target A row is DATA; this is documented as a dataset-design limitation rather than a classifier-skill result and motivates the explicit recommendation that future cohorts collect a configuration where DATA and ACK can come from the same beacon.

The LH4-G stratified-accuracy analysis confirmed that the CNN's headline lead is concentrated where it matters: in the highest-SNR test bucket (centered at 13.0 dB, $n = 81$, dominated by SF9 and SF10 rows where adjacent-SF discrimination

TABLE VIII
MODULE 4 TEST ACCURACY WITH 95% WILSON CONFIDENCE INTERVALS AT $n = 327$ TEST ROWS (SF, BEACON, PKT). THE MOD TASK IS SINGLE-CLASS IN THE DATASET AND WAS SKIPPED.

Model	sf	beacon	pkt
Random forest	0.966 [0.941, 0.981]	1.000	1.000
SVM (RBF, $C = 10$)	0.963 [0.937, 0.979]	1.000	1.000
MLP ($F \to 64 \to 32 \to C$)	0.966 [0.941, 0.981]	1.000	1.000
1-D CNN (8-frame seq.)	**0.985 [0.965, 0.993]**	1.000	1.000
Dummy (majority class)	0.725	0.905	0.905

TABLE IX
MODULE 4 LH4-F OPERATING-CONDITION SHIFT RESULT. RECOVERY IS VIA FINAL-LINEAR-ONLY FINE-TUNING OF THE CLEAN-TRAINED CNN BACKBONE ON 10–20% OF CONTENDED-TRAIN ROWS.

Task	Clean	Contended	Δ	Recovered
sf	0.978	0.000	+0.978	+0.304
beacon	1.000	0.696	+0.304	+0.000
pkt	1.000	0.696	+0.304	+0.000

is hardest), the CNN reached 0.938 against 0.914 / 0.901 / 0.914 for RF / SVM / MLP, while every model tied at 1.000 in the lower-SNR buckets dominated by the SF7 majority class. The active-station-count panel saw every model tie at 1.000 across both 1-station and 2-station test slices because the C5b ping-pong was deliberately staggered to minimize collisions. The distance panel could not be evaluated because C4 was not collected.

The LH4-F operating-condition shift experiment trained the CNN on the clean regime (single-station C1/C2) and evaluated on the contended regime (C5b ping-pong). Table IX reports the resulting accuracy gap Δ. The sf task collapsed completely on contended evaluation ($\Delta = +0.978$), and final-layer-only fine-tuning on 10–20% of contended training rows recovered only $+0.304$, falling short of the framework-specified $\geq$ half-of-Δ recovery target. Honest analysis of the result identified the dominant cause as a geometry change rather than the contended-regime channel effects the experiment was designed to isolate: the C5b session was conducted at a different physical separation from the C2 SF7 baseline, producing a clean-vs-contended RSSI distribution shift the clean-trained CNN never had a chance to generalize over. The intended fine-tune recovery requires a geometry-matched re-collection of C5 and likely a more aggressive unfreeze pattern (the FC head, not just the final linear layer); both are documented as Module 5 future-work items rather than reattempted under the present schedule.

The LH4-H deployment rubric scored each candidate model per task on accuracy (0.40), worst-bucket robustness (0.25), latency (0.15), interpretability (0.10), and complexity (0.10), with all numerical inputs read mechanically from the LH4-A through LH4-G manifests. The CNN won sf (the only discriminative task) with weighted total 4.40 against 1.65–1.95 for the row-models; its 8-frame sequence buffer captures the temporal context that gives it the worst-bucket robustness lead.

TABLE X
MODULE 4 LH4-H MODEL-SELECTION LATENCY AND ACCURACY. LATENCY IS PER FULL THREE-TASK PREDICT() CALL ACROSS 1,000 WARM ITERATIONS.

Model	p50 (ms)	p99 (ms)	In budget?	sf acc.
Random forest	224.3	289.6	**No (36× over)**	0.966
SVM (RBF)	0.541	0.861	Yes	0.963
MLP	0.438	0.856	Yes	0.966
1-D CNN	0.773	1.263	Yes	**0.985**

SVM won beacon and pkt on tiebreakers (lowest median latency, higher interpretability) since all four models tie at 1.000 accuracy on the alias. Random forest was disqualified on every task by an inference latency p99 of 289.6 ms—approximately 36× the 8 ms budget—driven by 200-tree predict_proba calls across three tasks per inference call. The 1,000-call warm microbenchmark per candidate is summarized in Table X, paired with the headline accuracy on the sf task.

The end-to-end deployment was validated on physical hardware. The deployed pipeline (CNN for sf; SVM for beacon and pkt) was streamed against the live FPGA UART export with both Targets ping-ponging at SF7 for a 3-minute timed run. The result was 362 frames classified across 180.7 s (2.00 fps, matching the configured 1 Hz cadence per Target × 2 Targets = 2 fps aggregate), every frame correctly classified, with median host inference latency of 1.00 ms and p99 of 1.60 ms. The 8 ms LH4-H budget was never threatened. Random forest, had it not been disqualified at design time, would have produced a ∼224 ms median latency in production—more than two orders of magnitude over budget—validating the rubric's latency-as-disqualifier rule on real hardware.

E. Module 5 Specification Status

Module 5 (system validation and benchmarking) is fully specified across LH5-A through LH5-H and summarized in §III.C: a four-dimensional stress-test matrix (S1 station density, S2 dynamic SF tracking, S3 low-SNR sweep, S4 30-minute endurance), a six-stage GPIO-toggle latency decomposition with uncertainty propagation, the S5 closed-loop PAUSE behavioral-response stress test, a six-point benchmarking matrix with regression rules, the empirically-derived single-page system specification sheet, the cross-domain failure-mode catalog with chain-depth attribution, and the curriculum-package and presentation deliverables. Cohort execution of these handouts on hardware was not performed under the present project's schedule and is left as future work; the four cross-domain integration faults already surfaced organically during Modules 1–4 (cataloged in §III.D) constitute the partial empirical evidence that the framework's failure-mode catalog approach produces real cross-domain attributions when executed.

F. Comparison to Single-Domain LoRa Spreading-Factor Classification

A direct numerical comparison against the closest single-domain prior work is informative even though the framework's

contribution is categorically a curriculum rather than a classifier. Mutescu et al. [5] report 99.77% test accuracy on LoRa spreading-factor detection across SF7–SF12 using a CNN over spectrogram inputs, with their dataset spanning a -20 dB to +30 dB SNR range. The framework's CNN reaches 0.985 (95% Wilson CI [0.965, 0.993]) on SF7–SF10 over the per-station feature vectors emitted by the Module 2 FPGA pipeline at $n = 327$ test rows. The two numbers are not directly comparable because the experimental conditions differ on three pedagogically informative axes. First, the framework's dataset spans only SF7–SF10 (SF11 and SF12 require three additional SX1276 register configurations not present in the LH1-C bring-up, and were deferred under schedule); the four-class problem is mechanically easier than the six-class problem the cited work solves. Second, the framework's dataset is single-modality (LoRa) and single-distance (3 m); C3 (modulation) was removed from the curriculum and C4 (distance) was deferred, removing two axes of intra-class variation present in the cited dataset. Third, the framework's CNN consumes per-emitter feature vectors (RSSI, SNR, inter-arrival time, ...) from the FPGA rather than spectrograms, so the model's inputs are categorically different from the cited work's inputs. The honest reading is that the framework's accuracy is competitive on a more constrained variant of the task using inputs an order of magnitude smaller than spectrograms, and that the dataset-scope limitations are themselves part of the framework's pedagogical evidence: each missing axis is a documented decision (C3 removed) or constraint (C4 deferred, SF11/12 abandoned) that students would close in a more complete cohort execution.

V. Conclusion

We have developed an integrated educational framework that treats the cross-domain interfaces between FPGA digital design, RF communications, embedded firmware, and machine learning as the primary teaching objects. The framework is realized as a five-module progressive curriculum paired with a benchtop platform of three STM32 Nucleo-L476RG nodes, RFM95W LoRa transceivers, and a Digilent Nexys A7-100T FPGA. The first four modules were executed end-to-end on physical hardware: the Module 2 ingestion pipeline closes timing at 100 MHz with all 105,975 endpoints meeting setup and hold and passes ten verification testbenches including byte-for-byte agreement with an independent Python reference across three recorded captures; Module 3 collected a 2,197-row labeled dataset across the executable subset of the planned campaigns; and Module 4 trained four classifier families per task with the 1-D CNN reaching 0.985 test accuracy on the SF task and a deployed three-task pipeline running live against the FPGA at 2.00 fps with 1.6 ms 99th-percentile inference latency. Four cross-domain integration faults surfaced organically during execution—a missing NVIC interrupt enable, a synthesis-vs-simulation reset divergence that converted block RAM into discrete flip-flops, a missing PA register write that broke link-budget monotonicity, and a session-confounded frequency-error feature—and constitute partial empirical evidence that the framework's integration surface produces the multidisciplinary reasoning artifacts it is designed to teach. The framework's CNN reaches 0.985 against the 0.9977 reported by single-domain prior work [5], achieved on a more constrained four-class variant of the spreading-factor task using per-emitter feature vectors rather than spectrograms. Cohort execution of the fully specified Module 5 stress, benchmarking, and failure-catalog program is left as future work, along with re-collection of the deferred distance and modulation campaigns and a geometry-matched contended regime to fully evaluate the operating-condition shift recovery target.

References

[1] H. Komkov, L. Pocher, A. Restelli, B. Hunt, and D. Lathrop, "RF signal classification using Boolean reservoir computing on an FPGA," in *Proc. 2021 Int. Joint Conf. Neural Networks (IJCNN)*, Jul. 2021, doi: 10.1109/IJCNN52387.2021.9533342.

[2] J. F. Lin, T. Morehouse, C. Montes, E. Caushi, A. Dudko, E. Savage, and R. Zhou, "A study of angle of arrival estimation of a RF signal with FPGA acceleration," in *Proc. 2024 Int. Wireless Commun. Mobile Comput. Conf. (IWCMC)*, 2024, pp. 1625–1628, doi: 10.1109/IWCMC61514.2024.10592551.

[3] A. Maclellan, L. H. Crockett, and R. W. Stewart, "Streaming convolutional neural network FPGA architecture for RFSoC data converters," in *Proc. 2023 21st IEEE Interreg. NEWCAS Conf. (NEWCAS)*, Edinburgh, UK, Jun. 2023, doi: 10.1109/NEWCAS57931.2023.10198198.

[4] K. A. Alnajjar, S. Ghunaim, and S. Ansari, "Automatic modulation classification in deep learning," in *Proc. 2022 5th Int. Conf. Commun., Signal Process., Appl. (ICCSPA)*, Cairo, Egypt, Dec. 2022, doi: 10.1109/ICCSPA55860.2022.10019122.

[5] P. M. Mutescu, A. Lavric, A. I. Petrariu, and V. Popa, "Deep learning enhanced spectrum sensing for LoRa spreading factor detection," in *Proc. 2023 13th Int. Symp. Adv. Topics Elect. Eng. (ATEE)*, Bucharest, Romania, Mar. 2023, doi: 10.1109/ATEE58038.2023.10108224.

[6] Y. E. Sagduyu and T. Erpek, "Adversarial attacks on LoRa device identification and rogue signal detection with deep learning," in *Proc. MILCOM 2023 — 2023 IEEE Military Commun. Conf.*, Boston, MA, Oct.–Nov. 2023, pp. 385–390, doi: 10.1109/MILCOM58377.2023.10356224.

[7] T.-P. Dang, T.-K. Tran, T.-T. Bui, and H.-T. Huynh, "LoRa gateway based on SoC FPGA platforms," in *Proc. 2021 Int. Symp. Elect. Electron. Eng. (ISEE)*, 2021, pp. 48–52, doi: 10.1109/ISEE51682.2021.9418711.

[8] K. Dakic, B. Al Homssi, A. Al-Hourani, and M. Lech, "LoRa signal demodulation using deep learning, a time-domain approach," in *Proc. 2021 IEEE 93rd Veh. Technol. Conf. (VTC2021-Spring)*, Apr. 2021, doi: 10.1109/VTC2021-Spring51267.2021.9448711.

[9] W. Smith, Z. Driskill, J. Goeders, and M. Wirthlin, "Digital design education using an open-source, cloud-based FPGA toolchain," in *Proc. 2024 Intermountain Eng., Technol. Comput. Conf. (IETC)*, May 2024, doi: 10.1109/IETC61393.2024.10564285.

[10] N. O. Strelkov, V. V. Krutskikh, and E. V. Shalimova, "Programming STM32 Nucleo platform for IoT education using STM32duino and Mbed OS," in *Proc. 2022 VI Int. Conf. Inf. Technol. Eng. Educ. (Inforino)*, Apr. 2022, doi: 10.1109/Inforino53888.2022.9782920.

[11] Digilent Inc., "Nexys A7 FPGA Board Reference Manual," Digilent Inc., Pullman, WA, 2023. [Online]. Available: https://digilent.com/reference/programmable-logic/nexys-a7/reference-manual

[12] Adafruit Industries, "Adafruit RFM69HCW and RFM9X LoRa Packet Radio Breakouts," Adafruit Industries, New York, NY, 2023. [Online]. Available: https://learn.adafruit.com/adafruit-rfm69hcw-and-rfm96-rfm95-rfm98-lora-packet-padio-breakouts/downloads

[13] STMicroelectronics, "STM32 Nucleo-64 Boards (MB1136) User Manual," UM1724, STMicroelectronics, Geneva, Switzerland, 2023. [Online]. Available: https://www.st.com/en/evaluation-tools/nucleo-l476rg.html

Appendix A
Project Repository

The complete project repository—containing all source code, lesson plans, lab handouts, FPGA RTL, host-side machine learning scripts, and supporting datasets used in this work—is available at https://github.com/JEschete/cross-domain-rf-ml.git.

Appendix B
Hardware Wiring Diagram

Figure 1 shows the per-node wiring used in the deployed benchtop platform. The Threat (interceptor) node carries the full three-way wiring: an STM32 Nucleo-L476RG paired with an RFM95W LoRa transceiver over SPI1, and a single-wire 3.3 V LVCMOS UART link from STM32 `USART3 TX` (`PC4`) to the Digilent Nexys A7-100T at PMOD JA pin 1 (FPGA pin `C17`), with common ground on PMOD JA pin 5. Each cooperative target node (Target A and Target B) uses the identical STM32+RFM95W wiring as the Threat, minus the FPGA UART link, since the targets do not export captured frames to the FPGA.

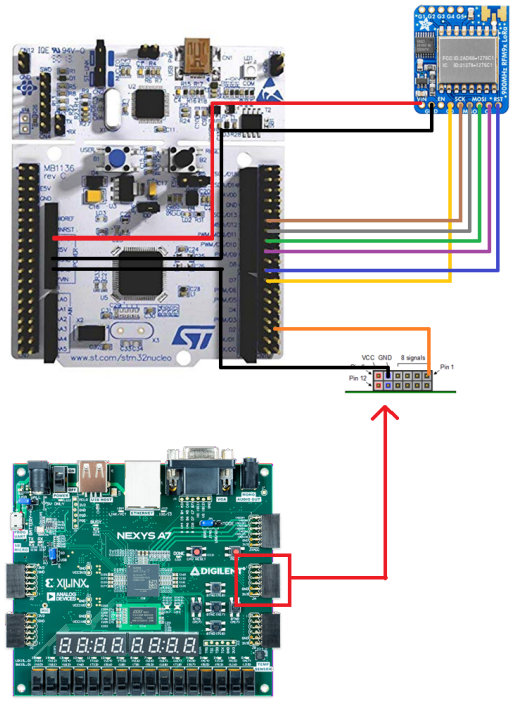

Fig. 1. Per-node wiring for the benchtop platform. The Threat node wires the STM32 to both the RFM95W (SPI) and the FPGA (UART over PMOD JA). Target A and Target B use the same STM32-to-RFM95W wiring without the FPGA link.

Appendix C
Lab Handout 2-H: Resource Utilization, Timing Analysis, and Bitstream Sign-Off

Lab Handout 2-H is provided as a representative example of the lab handouts that make up the curriculum. The complete handout from Module 2 is reproduced verbatim on the following pages. It closes Module 2 by running the full `ingest_top` design through Vivado implementation, analyzing the resource and timing reports, fixing any remaining negative slack, and producing a signed-off bitstream ready for Module 3 system integration.

Lab Handout 2-H

Resource Utilization, Timing Analysis, and Bitstream Sign-Off

Capstone — Module 2: Field-Programmable Gate Array (FPGA) Register-Transfer Level (RTL) Design and Signal Ingestion

---

1 Objective

Close Module 2 by running the full `ingest_top` design through Vivado implementation, analyzing the resource and timing reports, fixing any remaining negative slack, and producing a signed-off bitstream ready for Module 3 system integration.

2 Prerequisites

- Lab Handout 2-G (end-to-end simulation passing all three scenarios).

3 Procedure

3.1 Step 1 — Run implementation

1. Flow Navigator → **Run Implementation**. Wait for completion.
2. If synthesis or implementation errors appear, fix them before proceeding. Warnings about unused signals are acceptable; warnings about inferred latches are not.
3. Open the **Utilization** report. Record LUT, FF, Block Random Access Memory (BRAM), and Digital Signal Processing (DSP) usage.

Expected utilization envelope

For the reference implementation on the Artix-7 100T, expect roughly: LUT 1–3 %, FF 0.5–2 %, BRAM 2 tiles, DSP 0. Numbers substantially higher than this usually point at an accidental block-RAM inference or a combinational feedback loop.

3.2 Step 2 — Timing report

1. Open **Report Timing Summary**. Worst Negative Slack (WNS) must be ≥ 0 for both setup and hold.
2. If setup slack is negative, pull up the top 10 failing paths. Typical culprits: the Cyclic Redundancy Check (CRC) XOR tree (see 2-E), a long combinational compare in the parser Finite State Machine (FSM), or an unregistered output into `led`.
3. Register any failing output. Re-run implementation and confirm positive slack.

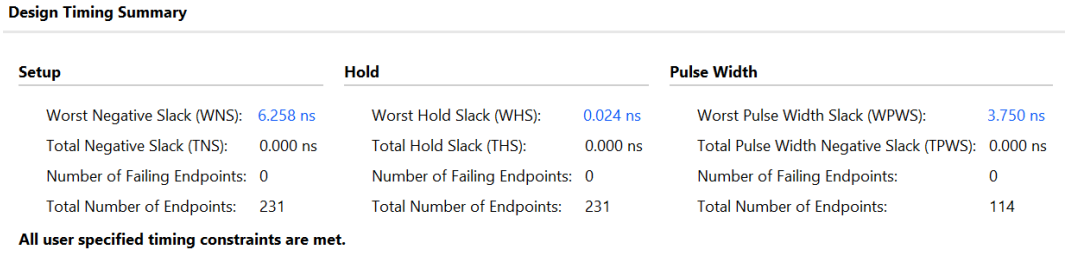

Figure 1: Vivado Timing Summary report with all four categories (WNS/TNS, WHS/THS, WPWS/TPWS) showing positive slack.

3.3 Step 3 — Clock domain crossings

Verify that Vivado's **Report CDC** shows only the expected crossing: `uart_rxd` into `sysclk` via `sync_2ff`. If any unexpected crossings appear, trace them before proceeding.

3.4 Step 4 — Generate the bitstream

1. **Confirm `ingest_top` is set as the Design Sources top.** In the Sources panel's *Design Sources* tab, `ingest_top` must appear in **bold**. If anything else is bold (a testbench from a recent simulation run, for example), right-click `ingest_top` → **Set as Top** before generating the bitstream.
2. Click **Generate Bitstream**. The output is `ingest_top.bit`.
3. Open the **Power** report. Dynamic power should be under 200 mW for this design.

3.5 Step 5 — Hardware sanity check

1. Program the Nexys A7 from Hardware Manager.
2. Connect the STM32 interceptor Universal Asynchronous Receiver/Transmitter (UART) `TX` to the Pmod JA pin declared in `ingest_top.xdc`.
3. Power on the cooperative-target pair (Target A and Target B) plus the interceptor (Threat). Per the `ingest_top` LED map established in LH2-F: `LED(0)` (UART byte strobe) flickers rapidly, `LED(1)` (`frame_valid`) pulses at the per-frame cadence, `LED(2)` (First In, First Out (FIFO) empty) is on most of the time, `LED(3)` (FIFO full) stays dark, `LED(4)` (`frame_reject`) stays dark on a clean stream, and `LED(5)` (`feat_busy`) flickers briefly during each feature update. `LED(15:8)` shows the selected feature byte (driven by `SW(7:0)` = row, `SW(12:8)` = byte index).
4. Add an Integrated Logic Analyzer (ILA) probe on `frame_valid` and `frame_reject`. Confirm that `frame_valid` pulses at roughly the Module 1 ping-pong cadence (DATA → ACK pairs) and `frame_reject` is rare.

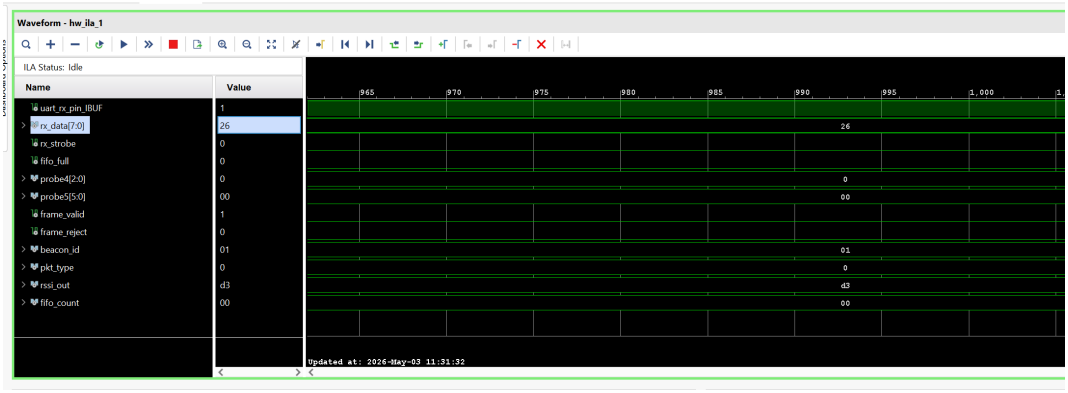

Figure 2: ILA capture on hardware: `frame_valid` pulses at the cooperative-target ping-pong cadence while `frame_reject` stays quiet.

3.6 Step 6 — Export reports to machine-readable format

The Module 5 system specification table (and the Module 2 resource row in the final report) requires exact LUT/FF/BRAM/WNS numbers. Export them from Vivado's Tcl console now so they are available without re-opening the GUI later.

```
# Open the implemented design if not already open
open_run impl_1

# Export utilization to CSV
report_utilization -hierarchical -file impl/util.rpt

# Export timing summary
report_timing_summary -file impl/timing.rpt -warn_on_violation

# Export power
report_power -file impl/power.rpt
```

Listing 1: Run in the Vivado Tcl Console after implementation completes.

Then run the following Python snippet once to extract the numbers into a structured `impl_summary.json` that the Module 5 spec-table script can read directly:

```
import json, re
from pathlib import Path

def parse_util(rpt: str) -> dict:
    patterns = {
        "lut":  r"Slice LUTs\s*\|\s*(\d+)",
        "ff":   r"Slice Registers\s*\|\s*(\d+)",
        "bram": r"Block RAM Tile\s*\|\s*(\d+)",
        "dsp":  r"DSPs\s*\|\s*(\d+)",
    }
    result = {}
    for key, pat in patterns.items():
        m = re.search(pat, rpt)
        result[key] = int(m.group(1)) if m else None
    return result

def parse_timing(rpt: str) -> dict:
    m = re.search(r"WNS\(ns\)\s+TNS\(ns\).*?\n\s*([-\d.]+)\s+([-\d.]+)",
        rpt, re.S)
    return {"wns_ns": float(m.group(1)) if m else None,
            "tns_ns": float(m.group(2)) if m else None}

util = parse_util(Path("util.rpt").read_text(errors="replace"))
timing = parse_timing(Path("timing.rpt").read_text(errors="replace"))

summary = {"module": "Module2_ingest_top", **util, **timing}
Path("impl_summary.json").write_text(json.dumps(summary, indent=2))
print(json.dumps(summary, indent=2))
```

Listing 2: `parse_vivado_reports.py` — run from the `impl/` directory.

Example output:

```
{
  "module": "Module2_ingest_top",
  "lut": 312,
  "ff": 198,
  "bram": 2,
  "dsp": 0,
  "wns_ns": 2.14,
  "tns_ns": 0.0
}
```

Commit `impl_summary.json` alongside the bitstream. This file is the authoritative source for Table 2 in the final report.

3.7 Step 7 — Archive the sign-off artifacts

Collect into a single directory:

- `ingest_top.bit`.
- `impl_summary.json` (from Step 6).
- The raw `util.rpt`, `timing.rpt`, and `power.rpt` files.
- The Module 2-G regression results (diff files from all three scenarios).
- A one-page sign-off summary confirming LUT/FF/BRAM within the expected envelope, WNS ≥ 0, and dynamic power < 200 mW.

4 Verification Checklist

- ☐ Implementation completes with zero errors and no inferred-latch warnings.
- ☐ WNS and WHS both ≥ 0.
- ☐ Report CDC shows only the `uart_rxd` → `sysclk` crossing.
- ☐ Dynamic power < 200 mW.
- ☐ Hardware ILA capture confirms `frame_valid` activity on a live cooperative-target ping-pong stream (with PAUSE bursts visible when the Threat user button is held).

Common Pitfalls

- **Negative hold slack**: almost always a missing register between two ultra-short paths. Add a single flip-flop rather than trying to slow the clock.
- **Inferred latch warning**: an incomplete `if`/`case` in a combinational process. Provide default assignments at the top of every combinational block.
- **Unexpected BRAM inference**: the parser's 32-byte `payload` register array accidentally sized above Vivado's distributed-RAM threshold. Force distributed RAM with the `(*ram_style = "distributed"*)` attribute.

Lab Handout 2-H Deliverable

1. `ingest_top.bit` and raw post-implementation reports (`util.rpt`, `timing.rpt`, `power.rpt`).
2. `impl_summary.json` with LUT/FF/BRAM/WNS fields populated (this feeds Table 2 of the final report directly).
3. One-page sign-off summary confirming all fields within spec.
4. ILA screenshot from live hardware.

Glossary

BRAM	Block Random Access Memory	1, 4, 5, 6
CRC	Cyclic Redundancy Check	2
DSP	Digital Signal Processing	1
FIFO	First In, First Out	3
FPGA	Field-Programmable Gate Array	1
FSM	Finite State Machine	2
ILA	Integrated Logic Analyzer	3, 6
RTL	Register-Transfer Level	1
UART	Universal Asynchronous Receiver/Transmitter	3